

API Testing with Postman

Тестування API за допомогою Postman — колекція запитів, змінні, автотести та Collection Runner

Автор: Анна Гуртова (Anna Hurtova)

Курс: Beetroot Academy / Helvetas Ukraine ("Стійкість")

Роль: Trainee QA Manual Engineer

Дата: Серпень 2026

Структура колекції «Homework 17»

Інструмент: Postman | **Мета:** Проектування та автоматизація колекції запитів для REST API (POST, GET, PATCH, DELETE) із використанням наскрізних змінних, позитивних/негативних сценаріїв та автотестів.

Колекція складається з **8 послідовних запитів**, об'єднаних у E2E-ланцюжок даних (Request Chaining):

POST Create → GET User → PATCH Edit → DELETE User → POST Neg → GET Neg → PATCH Neg → DELETE Neg

#	ЗАПИТ	МЕТОД	ТИП	ЗМІННІ
1	Create User	POST	Positive	{{user_name}} {{user_city}} → Body
2	GET User	GET	Positive	{{user_id}} → Params
3	EDIT User	PATCH	Positive	{{user_id}} → Body
4	DELETE User	DELETE	Positive	{{user_id}} → Params
5	Create User (Negative)	POST	Negative	{{user_name_negative}} → Body
6	GET User (Negative)	GET	Negative	Невалідний ID → Params
7	EDIT User (Negative)	PATCH	Negative	{{user_id_negative}} → Body
8	DELETE User (Negative)	DELETE	Negative	{{user_id_negative}} → Params

Позитивні запити та логіка змінних

POST Create User Positive

URL: `em.staging.api.onereach.ai/.../users`

Змінні в Body: `{{user_name}}` = yelyzaveta, `{{user_city}}` = Kyiv

Body (JSON):

```
{
  "name": "{{user_name}}",
  "age": "38",
  "gender": "female",
  "city": "{{user_city}}"
}
```

Pre-request Script:

```
pm.globals.set("user_name", "yelyzaveta");
pm.globals.set("user_city", "Kyiv");
```

Post-response (Request Chaining):

```
pm.globals.set("user_id", pm.response.json().id);
```

Автотести:

- ✓ Status code is 200 – перевірка успішного створення
- ✓ Response time < 5000ms – контроль часу відгуку

GET User Positive

URL: `/users?id={{user_id}}`

Змінні в Params: Key: id, Value: `{{user_id}}` (підхоплений з POST)

Автотести:

- ✓ Status code is 200 – підтвердження знаходження сутності
- ✓ Body matches string – валідація наявності "yelyzaveta" у відповіді

PATCH Edit User **Positive**

Змінні в Body: `{{user_id}}` → передається у JSON для ідентифікації сутності

Body (JSON):

```
{
  "id": "{{user_id}}",
  "name": "yelyzaveta3",
  "age": "38",
  "gender": "female",
  "city": "Kyiv"
}
```

Обґрунтування PATCH замість PUT: Під час тестування виявлено, що метод PUT повертав помилку «No user found». Оскільки за REST-стандартами PATCH використовується для часткової модифікації полів, запит переведено на PATCH — це приклад практичного дослідження особливостей API.

Автотести:

- ✓ Status code is 200 — успішне оновлення
- ✓ Body matches string — перевірка оновленого значення "yelyzaveta3"

DELETE User **Positive**

URL: `/users?id={{user_id}}`

Логіка: Видалення створеної сутності для збереження чистоти тестового середовища.

Автотести:

- ✓ Status code is 200 — підтвердження видалення
- ✓ Response time < 3000ms — контроль часу відгуку

Негативні запити (обробка помилок API)

POST Create User **Negative**

Суть: Передача неіснуючої змінної `{{user_name_negative}}` у Body.

Результат: Сервер заблокував створення — **400 Bad Request**, повідомлення «Validation error: name is invalid».

Автотести:

- ✓ Status code is 400 — верифікація захисного коду помилки
- ✓ Body matches string — валідація "Validation error: name is invalid"

GET User **Negative**

Суть: Передано невалідний ID у Query-параметрах: `?id=89187b8b-42NEGATIVE`.

Результат: Сервер повертає **200 OK**, проте у тілі відповіді — «No users found» (особливість архітектури вебсервісу).

Автотести:

- ✓ Body matches string — валідація наявності "No users found"

PATCH**Edit User**

Negative

Суть: Передача невалідного `{{user_id_negative}}` у Body.

Результат: Сервер відхилив оновлення — **200 OK**, повідомлення «No user found».

Автотести:

- ✓ Status code is 200 — верифікація статус-коду
- ✓ Body matches string — валідація "No user found"

DELETE**User**

Negative

Суть: Спроба видалення за невалідним `{{user_id_negative}}` у Params.

Результат: Сервер заблокував операцію — **200 OK**, повідомлення «No user found».

Автотести:

- ✓ Status code is 200 — верифікація статус-коду
- ✓ Body matches string — валідація "No user found"

Результати запуску (Collection Runner)

Для перевірки стабільності всього тестового сценарію виконано запуск через Collection Runner. Порядок: спочатку позитивний CRUD-сценарій, потім негативні перевірки — це дозволило позитивним крокам створити та передати по ланцюжку реальний `{{user_id}}`.

Collection Runner — Homework 17

15 Total Tests **15** Passed **0** Failed **0** Errors

Примітка: Сумарна кількість тестів становить 15 (а не 16), оскільки у Кроці 6 (GET Negative) перевірку сфокусовано на одному комплексному автотесті валідації тексту помилки сервера.